

Examen d'entraînement LEPL1110

Éléments Finis

Équipe pédagogique — Mai 2026

Durée : 3 heures. Aucun document autorisé. Calculatrice non autorisée.

Les exercices sont indépendants et peuvent être traités dans n'importe quel ordre.

1 Équation de diffusion de la chaleur en 1D

Une barre métallique de longueur $L = 1$, de diffusivité thermique $\kappa > 0$, est thermiquement isolée à ses deux extrémités. La température $u(x, t)$ est solution de l'équation de diffusion :

$$\partial_t u - \kappa \partial_{xx} u = 0, \quad x \in (0, 1), \quad t > 0,$$

avec les **conditions aux limites de Neumann homogènes** (barres isolées) :

$$\partial_x u(0, t) = 0, \quad \partial_x u(1, t) = 0,$$

et la **condition initiale** : $u(x, 0) = u_0(x)$.

1.1 Formulation faible en espace

Multiplier l'équation de diffusion par une fonction test $v \in H^1(0, 1)$ et intégrer sur $(0, 1)$. En utilisant une intégration par parties, montrer que la formulation variationnelle s'écrit :

$$\int_0^1 \partial_t u v \, dx + \kappa \int_0^1 \partial_x u \partial_x v \, dx = 0, \quad \forall v \in H^1(0, 1).$$

Expliquer pourquoi les conditions aux limites de Neumann n'apparaissent pas explicitement dans cette formulation.

1.2 Semi-discrétisation en espace

On cherche une approximation de la forme $u_h(x, t) = \sum_{j=0}^N u_j(t) \varphi_j(x)$, où $\{\varphi_j\}$ est une base d'éléments finis P_1 sur un maillage de N éléments uniformes.

En choisissant $v = \varphi_i$ dans la formulation faible, montrer que le vecteur $\mathbf{u}(t) = (u_0(t), \dots, u_N(t))^T$ vérifie le **système différentiel** :

$$M \dot{\mathbf{u}}(t) + \kappa K \mathbf{u}(t) = \mathbf{0},$$

où la **matrice de masse** M et la **matrice de rigidité** K sont définies par :

$$M_{ij} = \int_0^1 \varphi_j \varphi_i \, dx, \quad K_{ij} = \int_0^1 \partial_x \varphi_j \partial_x \varphi_i \, dx.$$

1.3 Matrices élémentaires et assemblage ($N = 2$)

On discrétise la barre avec $N = 2$ **éléments** P_1 **uniformes** de longueur $h = 1/2$. Les nœuds sont situés en $x_0 = 0$, $x_1 = 1/2$, $x_2 = 1$.

(a) Calculer la **matrice de rigidité élémentaire** $K^{(e)}$ et la **matrice de masse élémentaire** $M^{(e)}$ pour un élément de longueur h .

Rappel : pour un élément $[x_e, x_{e+1}]$ de longueur h , les fonctions de forme P_1 sont $\varphi_0(x) = (x_{e+1} - x)/h$ et $\varphi_1(x) = (x - x_e)/h$.

(b) Assembler les **matrices globales** M et K de taille 3×3 .

(c) Vérifier que $K \mathbf{1} = \mathbf{0}$, où $\mathbf{1} = (1, 1, 1)^\top$. Donner une interprétation physique de cette propriété.

1.4 Schéma en temps (schéma θ)

On discrétise le temps avec un pas Δt , et on pose $t^n = n\Delta t$, $\mathbf{u}^n \approx \mathbf{u}(t^n)$.

(a) Écrire le **schéma** θ qui discrétise le système $M\dot{\mathbf{u}} + \kappa K\mathbf{u} = \mathbf{0}$. Pour quelles valeurs de θ obtient-on l'**Euler explicite**, l'**Euler implicite** et le schéma de **Crank–Nicolson** ?

(b) Pour le problème continu, montrer que $\frac{d}{dt} \|u(t)\|_{L^2}^2 \leq 0$. Pour quelle(s) valeur(s) de θ le schéma θ est-il **inconditionnellement** L^2 -stable ?

(c) Pour $\theta = 0$ (Euler explicite), exprimer la **condition de stabilité** sur Δt en fonction de la plus grande valeur propre $\lambda_{\max}$ du problème généralisé $K\phi = \lambda M\phi$.

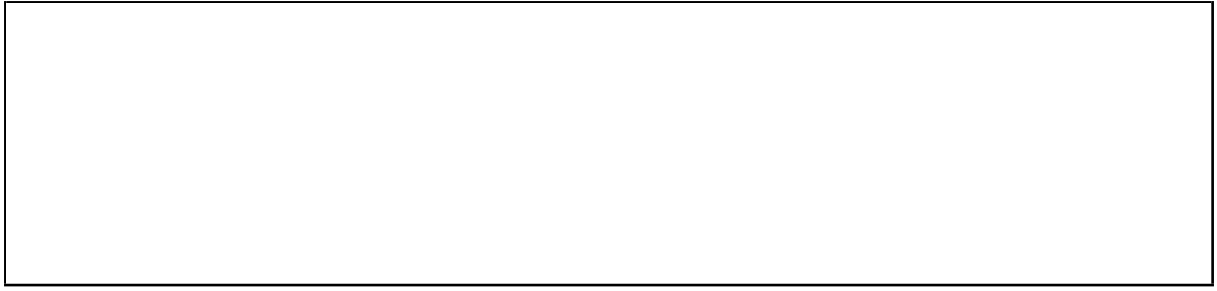
1.5 Analyse modale et solution exacte

(a) Trouver les **fonctions propres** $\phi_n(x)$ et les **valeurs propres** λ_n du problème :

$$-\phi_n'' = \lambda_n \phi_n \quad \text{sur } (0, 1), \quad \phi_n'(0) = \phi_n'(1) = 0.$$

(b) Pour la condition initiale $u_0(x) = \cos(\pi x)$, écrire la **solution exacte** $u(x, t)$.

(c) Quel est le **temps caractéristique de décroissance** τ du mode dominant (non constant) ? La température finit-elle par se stabiliser ? Si oui, vers quelle valeur limite ?



2 Le jeu des erreurs

La formulation par éléments finis de l'équation de diffusion de la chaleur a été implémentée en Python dans le code ci-dessous. On suppose une barre de taille 1, discrétisée avec N éléments P_1 uniformes. On utilise la quadrature de Gauss–Legendre à 2 points sur $[-1, 1]$.

Le code est censé retourner la matrice de masse M et la matrice de rigidité K . Les conditions aux limites ne sont pas imposées ici.

Ce code contient volontairement un maximum de sept erreurs.

Repérez ces erreurs en annotant le code à l'aide des numéros ① à (maximum) ⑦. Pour chacune d'elles, reportez-vous à la liste ci-dessous et fournissez un bref commentaire (une phrase maximum) expliquant la nature de l'erreur.

```

1 import numpy as np
2
3 def assemble_heat_1d(N, kappa):
4     x = np.linspace(0.0, 1.0, N)
5
6     M = np.zeros((N + 1, N + 1))
7     K = np.zeros((N + 1, N + 1))
8
9     xi_q = [-1.0 / np.sqrt(3.0), 1.0 / np.sqrt(3.0)]
10    w_q = [1.0, 1.0]
11
12    dN_dxi = [-0.5, 0.5]
13
14    for e in range(N):
15        x0 = x[e]
16        x1 = x[e + 1]
17        h = x1 - x0
18        Jac = h
19        dxi_dx = 1.0 / Jac
20
21        dof = [e, e + 1]
22
23        for q in range(2):
24            xi = xi_q[q]
25            w = w_q[q]
26
27            Nq = [0.0, 0.0]
28            Nq[0] = 0.5 * (1.0 + xi)
29            Nq[1] = 0.5 * (1.0 - xi)
30
31            dN_dx = [0.0, 0.0]
32            for i in range(2):
33                dN_dx[i] = dN_dxi[i] / dxi_dx
34
35            for i in range(2):
36                I = dof[i]
37                for j in range(2):
38                    J = dof[j]
```

```
39  
40         M[I, J] += Nq[i] * Nq[j] * w * Jac  
41         K[I, J] += kappa * dN_dx[i] * dN_dx[j] * Jac  
42  
43     return K, M
```

- ① _____
- ② _____
- ③ _____
- ④ _____
- ⑤ _____
- ⑥ _____
- ⑦ _____